

Practical 1: Setting Up and Configuring a Malware Analysis Laboratory (Kali Linux)

Step 1: Install Virtualization Software

- Install a virtualization platform such as VirtualBox or VMware Workstation on the host system.
- Ensure the host system supports hardware virtualization.
- Enable Intel VT-x or AMD-V virtualization in the BIOS/UEFI settings if not already enabled.
- Verify successful installation by launching the virtualization software.

Step 2: Create a Kali Linux Virtual Machine

- Create a new virtual machine in the virtualization software.
- Select Linux as the operating system type and Debian (64-bit) as the version.
- Allocate appropriate resources:
 - * RAM: Minimum 2 GB (4 GB recommended)
 - * CPU: 1–2 cores
 - * Disk space: 40 GB or more
- Attach the Kali Linux ISO file or use a prebuilt Kali Linux virtual image.
- Complete the Kali Linux installation and verify successful boot.

Step 3: Configure Network Isolation

- Open the virtual machine's network settings.
- Set the network adapter to Host-Only Adapter or Internal Network mode.
- Disable direct internet access to prevent malware from communicating externally.
- Verify isolation by attempting to access an external website from the VM.

Step 4: Configure Security and System Settings

- Disable automatic system updates to maintain a stable analysis environment.
- Stop or disable unnecessary background services that may interfere with malware execution.
- Configure firewall rules if needed to fully restrict outbound connections.
- Enable viewing of hidden files and file extensions for better analysis visibility.

Step 5: Install Malware Analysis Tools

- Process monitoring tools (htop, ps, strace)
- Network analysis tools (Wireshark, tcpdump)
- Static analysis tools (WxhexEditor, Detect-it-easy, strings, objdump, readelf, flare-floss)
- Binary analysis and reverse engineering tools (radare2, Ghidra)

Step 6: Create a Shared Folder

- Configure a shared folder between the host system and the Kali Linux VM.
- Use the shared folder strictly for transferring analysis tools or malware samples.
- Disable the shared folder after file transfer to reduce risk.
- Avoid enabling clipboard sharing or drag-and-drop features.

Step 7: Take a Clean Snapshot

- Shut down the Kali Linux virtual machine after completing configuration.
- Take a snapshot of the VM and name it "Clean State".
- Confirm that the snapshot can be restored successfully.
- Use this snapshot as a baseline before executing any malware samples.

PRACTICAL – 2

To perform basic static analysis of Windows malware binaries on a Linux platform

Step 1: Prepare the Analysis Environment

Boot into a Linux system (Ubuntu / Kali / Parrot).

Ensure internet connectivity is disabled (recommended).

Log in using a non-root user account.

Step 2: Create a Dedicated Malware Analysis Directory

CMD - mkdir -p ~/malware_analysis

cd ~/malware_analysis

This isolates malware samples from the rest of the system.

Step 3: Copy the Malware Sample

Copy the Windows malware binary into the analysis directory.

CMD - cp /path/to/malware.exe .

Verify the file exists:

CMD - ls -l

Step 4: Identify the File Type

Confirm that the file is a Windows executable.

CMD - file malware.exe

Expected Output: PE32 executable (GUI) Intel 80386, for MS Windows

Step 5: Generate Cryptographic Hashes

Generate MD5 Hash

CMD- md5sum malware.exe

Generate SHA-1 Hash

CMD - sha1sum malware.exe

Generate SHA-256 Hash

CMD - sha256sum malware.exe

Save Hash Values to a File

CMD- md5sum malware.exe > hashes.txt

sha1sum malware.exe >> hashes.txt

sha256sum malware.exe >> hashes.txt

Step 6: Extract Printable Strings (ASCII)

Extract human-readable strings from the malware binary.

CMD - strings malware.exe > strings.txt

Step 7: View Extracted Strings

CMD - less strings.txt

Step 8: Extract Unicode Strings

CMD- strings -el malware.exe > unicode_strings.txt

Step 9: Search for Suspicious Indicators

Step 9.1: Search for URLs

grep -i "http" strings.txt

Step 9.2: Search for IP Addresses

grep -Eo '([0-9]{1,3}\.){3}[0-9]{1,3}' strings.txt

Step 9.3: Search for Command Execution

grep -i "cmd" strings.txt

Step 9.4: Search for Registry Entries

grep -i "Software\\Microsoft" strings.txt

Step 9.5: Search for Suspicious Windows APIs

grep -iE "CreateProcess|WinExec|RegSetValue|WriteFile" strings.txt

Step 10: Identify Possible Obfuscation or Packing

Check if meaningful strings are missing.

head strings.txt

Practical 3: Malware Detection using YARA Rules

Aim

To create and apply YARA rules for identifying malware based on patterns and signatures.

Tools Required

- YARA
- Sample malware files / test files
- Text editor (Notepad++ / VS Code)

Step-by-Step Execution

Part A: Installation of YARA

1. **Download YARA**
 - Visit official GitHub page of YARA
 - Download latest release for your OS
2. **Install YARA**
 - Extract downloaded files
 - Add YARA folder to system **PATH** (for command-line use)
3. **Verify Installation**
 - Open Command Prompt
 - Type:

yara --version
 - Confirm installation is successful

Part B: Writing YARA Rules

4. **Create Rule File**
 - Open text editor
 - Save file as sample_rule.yar

5. Write Basic YARA Rule

```
rule Suspicious_String
{
  strings:
    $a = "malware"
    $b = "hack"
    $c = "cmd.exe"

  condition:
    any of them
}
```

6. Save the File

Part C: Applying YARA Rule

7. Run YARA Scan

- o Open Command Prompt
- o Execute:

```
yara sample_rule.yar suspicious_file.exe
```

8. Analyze Output

- o If matched → rule name will be displayed
- o If no match → no output

Part D: Advanced Rule (Optional)

9. Add File Size Condition

```
condition:
  filesize < 1MB and any of them
```

10. Use Multiple Conditions

- Combine strings and logic for accuracy

Practical 4: Understanding C Code Constructs in Assembly Language

Aim

To understand how high-level C constructs are translated into Assembly language.

Tools Required

- GCC Compiler
- Any editor (VS Code / Notepad++)
- Debugger (optional)

Step-by-Step Execution

Part A: Writing C Program

1. Create a C Program

```
#include <stdio.h>

int main() {
    int a = 5, b = 10, c;
    c = a + b;
    printf("%d", c);
    return 0;
}
```

2. Save File

- o Save as program.c

Part B: Convert C to Assembly

3. Compile to Assembly

- o Open terminal
- o Run:

gcc -S program.c

4. View Assembly File

- o Open generated file program.s

Part C: Analysis of Assembly Code

1. Sections in Assembly

- .rdata → Read-only data (constants like format string)
- .text → Executable instructions

Example:

```
.section .rdata,"dr"
```

LC0:

```
.ascii "%d\\0"
```

Stores format string for printf

2. Function Entry (Prologue)

```
pushl %ebp
```

```
movl %esp, %ebp
```

```
subl $32, %esp
```

Explanation:

- pushl %ebp → Save old base pointer
- movl %esp, %ebp → Create new stack frame
- subl \$32, %esp → Allocate 32 bytes for local variables

3. Variable Storage

```
movl $5, 28(%esp) ; a = 5
```

```
movl $10, 24(%esp) ; b = 10
```

Variables are stored in stack memory relative to %esp

Variable	Location
----------	----------

a	28(%esp)
---	----------

b	24(%esp)
---	----------

Part D: Stack and Registers

Important Registers

Register	Purpose
----------	---------

%esp	Stack pointer
------	---------------

%ebp Base pointer

%eax Return value / calculations

%edx Temporary storage

Practical 6: Malware Analysis using IDA Pro (Disassembly & Reverse Engineering)

1. Aim

To analyze a malware executable using IDA Pro and understand its internal structure, functions, and behavior through disassembly.

2. Requirements

IDA Pro (IDA Free version)

Malware sample (sample.exe)

Windows system

3. Pre-Lab Preparation

3.1 Installation Details

Source of download: Official Hex-Rays website

Version installed: IDA Free 8.x

Installation status: ☒ Successful ☐ Not Successful

4. Procedure & Observations

Step 1: Launch IDA Pro

Observation:

IDA Pro was successfully launched and the malware sample sample.exe was loaded into the tool.

Step 2: Auto Analysis

Observation:

IDA Pro performed automatic disassembly and analysis. Functions, segments, and imports were identified.

Step 3: Entry Point Analysis

Entry Point Address: 0x401000

Observation:

The entry point contains initialization instructions and calls to system APIs. Control is transferred to the main function after setup.

Step 4: String Analysis

S.No

String Value

Suspicion

Remarks

1

<http://malicious-site.com>

Yes

Possible C2 communication

2

cmd.exe /c

Yes

Executes system commands

3

powershell.exe -nop -enc

Yes

Obfuscated script execution

4

C:\Windows\System32

No

Common system path

5

Run\Microsoft\Windows\CurrentVersion

Yes

Persistence via registry

Step 5: Cross-Reference Analysis

Observation:

The suspicious URL string is referenced in a function responsible for network communication. The cmd.exe string is used in a function that executes system commands.

Step 6: Control Flow Analysis

Observation:

Graph view shows multiple conditional branches and loops. One loop repeatedly attempts network connection, indicating beaconing behavior.

Step 7: API Call Analysis

S.No

API Function

Purpose

1

CreateFileA

Creates or opens files

2

WriteFile

Writes data to file

3

RegSetValueExA

Modifies registry (persistence)

4

WinExec

Executes external programs

Step 8: Function Identification

Original Name

Renamed Function

Reason

sub_401200

network_connect

Handles server communication

sub_401350

execute_command

Executes system commands

sub_401500

registry_persistence

Adds autorun entry

Step 9: Behavioral Analysis

☒ File creation

☒ Registry modification

☒ Network communication

☒ Persistence mechanism